

CAPSTONE PROJECT

HandsMen Threads: Elevating the Art of Sophistication in Men's Fashion

In Partial Fulfillment of the Requirements in
Salesforce Internship

Presented to
Rakesh Bhoomani

By
Garces, Grace S.
(November 12, 2025)

Abstract

HandsMen Threads, a dynamic fashion organization, is implementing a Salesforce CRM solution designed to revolutionize data management and enhance customer relations. The project focuses on creating a robust data model to store all business data and maintain a seamless flow of information across departments. A major goal of the project is to maintain data integrity directly from the user interface (UI), ensuring that all data entered into the system is accurate, consistent, and reliable for decision-making. The solution integrates several automated processes aimed at improving customer engagement, operational efficiency, and business intelligence.

Objective

The primary objective of the HandsMen Threads Salesforce Project is to design and implement a custom CRM system that automates and manages the company's sales, customer service, and inventory workflows.

Specific Objectives:

- To build a customized data model tailored for the fashion retail industry.
- To ensure data quality and consistency through validation rules and UI-level constraints.
- To automate core business processes like order confirmations, loyalty program updates, and stock management.
- To provide real-time insights through dashboards and reports for informed decision-making.
- To use Salesforce Lightning for an intuitive and responsive user interface.

Technology Description

Platform: Salesforce CRM (Lightning Experience)

The project is built on Salesforce Lightning Experience, the modern, component-based interface of Salesforce. It provides a fast, user-friendly, and customizable environment that enhances productivity through improved navigation, responsive design, and seamless integration across desktop and mobile devices. This platform serves as the foundation for all system functionalities, ensuring smooth data management and scalability.

Automation Tools: Flows, Process Builder (Legacy Reference), Apex Triggers

Business automation in the project is achieved through a combination of Flows and Apex Triggers, with Process Builder referenced for older processes being transitioned.

- Flows handle declarative automation such as sending automated order confirmations, updating customer loyalty statuses, and triggering notifications.
- Apex Triggers manage more complex, record-level logic that requires custom conditions or multi-object updates.

- Process Builder serves as a legacy tool, with existing automations gradually being migrated to Flows for improved performance and maintainability.

Programming Language: Apex

Apex is Salesforce's proprietary programming language, designed for building custom logic and server-side processes within the Salesforce environment. It enables developers to perform advanced automation, complex data manipulation, and asynchronous operations such as scheduled batch processing. In this project, Apex is used to execute tasks like daily bulk order updates, loyalty recalculations, and integration with other Salesforce components.

Email Communication: Salesforce Email Templates

Automated emails are implemented using Salesforce Email Templates, which ensure professional, consistent, and branded communication. These templates dynamically merge relevant customer and order information, allowing for personalized messages such as order confirmations, loyalty updates, and stock alerts. This automation enhances customer engagement and operational efficiency.

Database: Salesforce Standard & Custom Objects

The project utilizes both Standard and Custom Objects to manage business data. Standard Objects like Account, Contact, and Product handle core CRM data, while Custom Objects such as Inventory, Orders, and Loyalty are designed specifically for HandsMen Threads' business processes. These objects are linked through relationships (lookup and master-detail) and validated with field-level rules to maintain data integrity and reliability.

User Interface: Lightning App Builder

The Lightning App Builder is used to design interactive and intuitive user interfaces through a drag-and-drop approach. It enables the creation of custom record pages, dashboards, and workspace layouts tailored to user roles. This customization improves user experience, streamlines navigation, and simplifies data entry within the Salesforce environment.

Testing and Deployment: Salesforce Sandbox and Change Sets

All development and testing activities are performed in a Salesforce Sandbox, a safe and isolated environment that prevents any impact on live data. Once the features are validated, deployment to the production environment is executed using Change Sets. This ensures that all updates, configurations, and code changes are migrated in a controlled, traceable, and error-free manner.

System Requirements

Supported Browsers

- Google Chrome (recommended)

- Mozilla Firefox
- Microsoft Edge
- Safari (Mac only)
- Internet Explorer 11 (limited support)

Note: At least two browsers should be installed.

Operating System Compatibility

- Windows 8 / 10 / 11
- macOS (latest versions)
- Linux (limited support, browser-dependent)
- ChromeOS (browser-based)

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4GB minimum (8GB recommended)
- Storage: 10GB free disk space
- Display: 1366x768 (1920x1080 recommended)

Network Requirements

- Stable broadband (minimum 30 Mbps)
- No VPN restrictions blocking Salesforce access
- Allow domains: .salesforce.com, .force.com

Solution Architecture Overview

The system architecture consists of three main layers:

- Data Layer – Built using Standard and Custom Salesforce Objects (Accounts, Contacts, Orders, Inventory, Loyalty).
- Logic Layer – Implements automation through Flows, Apex Triggers, and Scheduled Apex.
- Presentation Layer – Uses Lightning App Builder to create dynamic record pages and dashboards for different user roles.

Object Model

The Object Model defines the core data architecture of the HandsMen Threads Salesforce CRM implementation, integrating both Standard and Custom Objects to manage customer, product, order, inventory, and marketing data. This structure ensures smooth information flow across the system and supports automation for sales, inventory control, and campaign management.

The HandsMen_Customer__c object serves as the foundation for customer management. It is a Standard Object with key fields such as Account Name and Account Type. It has a parent relationship to Contacts and is primarily used to store company and individual customer information, serving as the main record for all client-related activities.

The HandsMen_Product__c object, also a Standard Object, stores all product details including Product Name, SKU, and Price. It is referenced in both Orders and Inventory objects. Its main purpose is to maintain an up-to-date catalog of items sold by HandsMen Threads, supporting pricing, sales tracking, and stock monitoring.

The HandsMen_Order__c object is a Custom Object that captures order-related data. Key fields include Order Number, Order Date, Total Amount, and Status. It has lookup relationships to both Account and Product, allowing each order to be tied to a specific customer and product. This object facilitates transaction management, order tracking, and payment computation.

The Inventory__c object, another Custom Object, contains fields such as Product, Stock Quantity, and Reorder Level, with a lookup relationship to the Product object. It is used to track stock availability and automate alerts when inventory levels fall below defined thresholds, ensuring timely restocking and smooth supply chain operations.

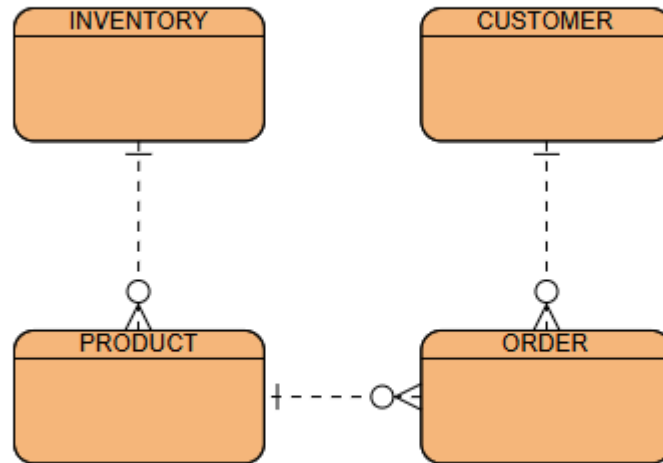
Finally, the Marketing_Campaign__c object is a Custom Object designed for managing promotional activities. It includes key fields like Marketing Campaign Number (Auto Number: MC-{0000}), Campaign Name, Start Date, and End Date. It can relate to Leads, Contacts, or Orders for performance tracking. This object helps HandsMen Threads plan, execute, and measure the effectiveness of marketing campaigns, promoting customer engagement and sales growth.

Entity Relationship Diagram (ERD)

Below is a conceptual representation of the data relationships:

- Each Customer may create multiple Orders, and conversely, each Order belongs to a single Customer.
- Each Order may contain many Products, and conversely, each Product can appear in multiple Orders.
- Each Product is recorded in Inventory, and conversely, each Inventory entry corresponds to one specific

Figure 1. Entity Relationships Diagram



The data model follows a relational structure where customers place orders, orders include multiple products, and products are tracked in the inventory. This ensures accurate order processing, efficient stock management, and clear traceability of customer transactions.

Detailed Explanation of Project Phases

Phase 1: System Planning and Design

The project began with identifying the needs of the fictional brand HandsMen Threads. The key entities—Customers, Products, Orders, and Inventories—were mapped and their relationships defined. Data fields, validation rules, and automation logic were documented before actual implementation.

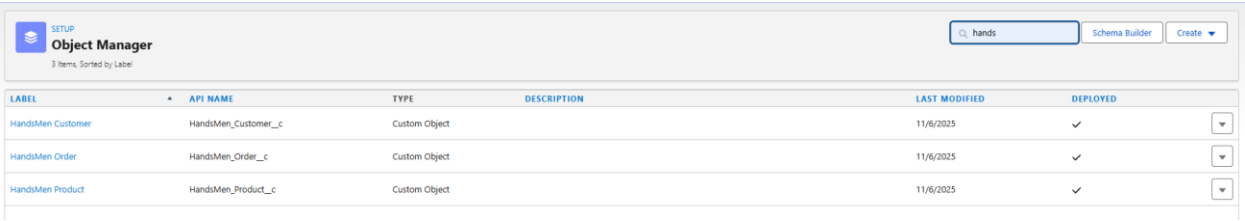
Phase 2: Custom Object Creation

Custom objects were created to handle specific data:

- HandsMen Customers – Stores customer details (name, email, contact, address, total purchases, loyalty status).
- HandsMen Products – Contains product details such as name, SKU, category, and price.
- HandsMen Orders – Tracks customer purchases, linked to both the customer and product.
- Inventory – Monitors stock availability and warehouse location.

- Marketing Campaign – Manages promotional activities and marketing initiatives, tracking campaign names, durations, target audiences, and performance metrics to analyze campaign effectiveness and customer engagement.

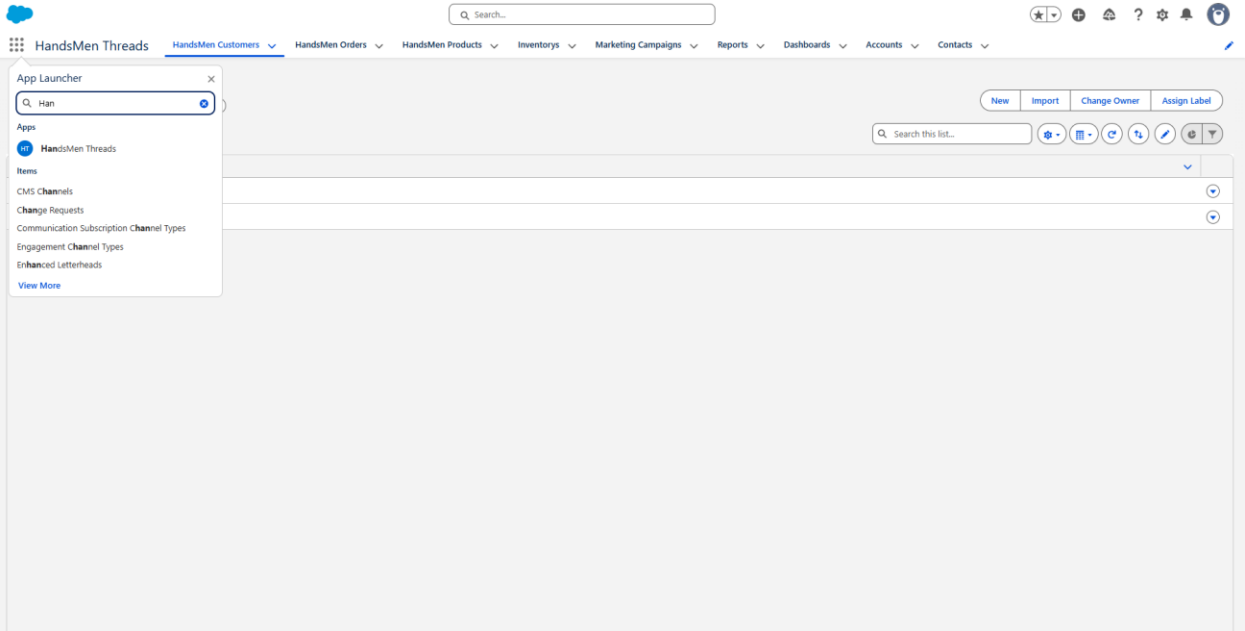
Figure 2.1. Custom Object Creation



The screenshot shows the Salesforce Object Manager interface. At the top, there's a header with 'SETUP Object Manager' and a search bar containing 'hands'. Below the header is a table with columns: LABEL, API NAME, TYPE, DESCRIPTION, LAST MODIFIED, and DEPLOYED. The table lists three custom objects: HandsMen Customer, HandsMen Order, and HandsMen Product, all created on 11/6/2025 and marked as deployed.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
HandsMen Customer	HandsMen_Customer_c	Custom Object		11/6/2025	✓
HandsMen Order	HandsMen_Order_c	Custom Object		11/6/2025	✓
HandsMen Product	HandsMen_Product_c	Custom Object		11/6/2025	✓

Figure 2.2. HandsMen Thread App Creation



Phase 3: Validation and Relationships

Validation rules were applied to ensure data accuracy (e.g., valid email formats).

Figure 3.1 Customer Email Validation Format

The screenshot shows the 'HandsMen Customer Validation Rule' configuration page. At the top, there's a title 'HandsMen Customer Validation Rule' with a 'Back to HandsMen Customer' link and a 'Help for this Page' icon. Below the title is a 'Validation Rule Detail' section with 'Edit' and 'Clone' buttons. The rule details are as follows:

Rule Name	Email	Active
Error Condition Formula	NOT CONTAINS(Email__c, "@gmail.com")	☒
Error Message	Please fill Correct Gmail	Error Location Top of Page
Description		
Created By	Grace Garces, 11/6/2025, 7:22 PM	Modified By Grace Garces, 11/6/2025, 7:22 PM

At the bottom of the rule details, there are 'Edit' and 'Clone' buttons.

Figure 3.2 Customer Email Validation Alert

The screenshot shows the 'New HandsMen Customer' form. The form has a header 'New HandsMen Customer' and a note '* = Required Information'. The form is divided into sections: 'Information', 'Owner', 'FirstName', 'LastName', and 'Total Purchases'. The 'Information' section contains fields for 'HandsMen Customer Name' (with value 'Luis'), 'Email' (with value 'luis@abc.com'), 'Phone', and 'Loyalty Status' (with value '--None--'). The 'Owner' section shows 'Grace Garces'. The 'FirstName' field has the value 'Luis Aldrin' and the 'LastName' field has the value 'Viray'. The 'Total Purchases' field is empty. A validation error alert is displayed at the bottom of the form, stating 'We hit a snag.' and 'Review the errors on this page.' with a list of errors: 'Please fill Correct Gmail'. The alert has a close button (X) and a link to 'Review the errors on this page.' At the bottom of the form, there are 'Cancel', 'Save & New', and 'Save' buttons.

Lookup and master-detail relationships were set to connect customers, orders, and products.

Figure 4. Customer Order Lookup Relationship

SETUP > OBJECT MANAGER

HandsMen Order

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Fields & Relationships

10 Items, Sorted by Field Label

Quick Find

NewDeleted FieldsField DependenciesSet History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Customer Email	Customer_Email__c	Email		
HandsMen Customer	HandsMen_Customer__c	Lookup(HandsMen Customer)		

Phase 4: Automation with Flows and Triggers

This phase focuses on implementing business process automation using Salesforce Flows and Apex Triggers. The goal is to reduce manual effort, ensure real-time data updates, and enhance operational efficiency across order management, inventory tracking, and customer engagement.

1. Order Confirmation Email (Record-Triggered Flow)

Trigger: When a new Order record is created or its status changes to “Confirmed.”

Action: Automatically sends a confirmation email to the customer using a predefined Salesforce Email Template.

Purpose: Keeps customers informed of their order status and improves communication efficiency.

Figure 5.1 Order Confirmation Email Flow

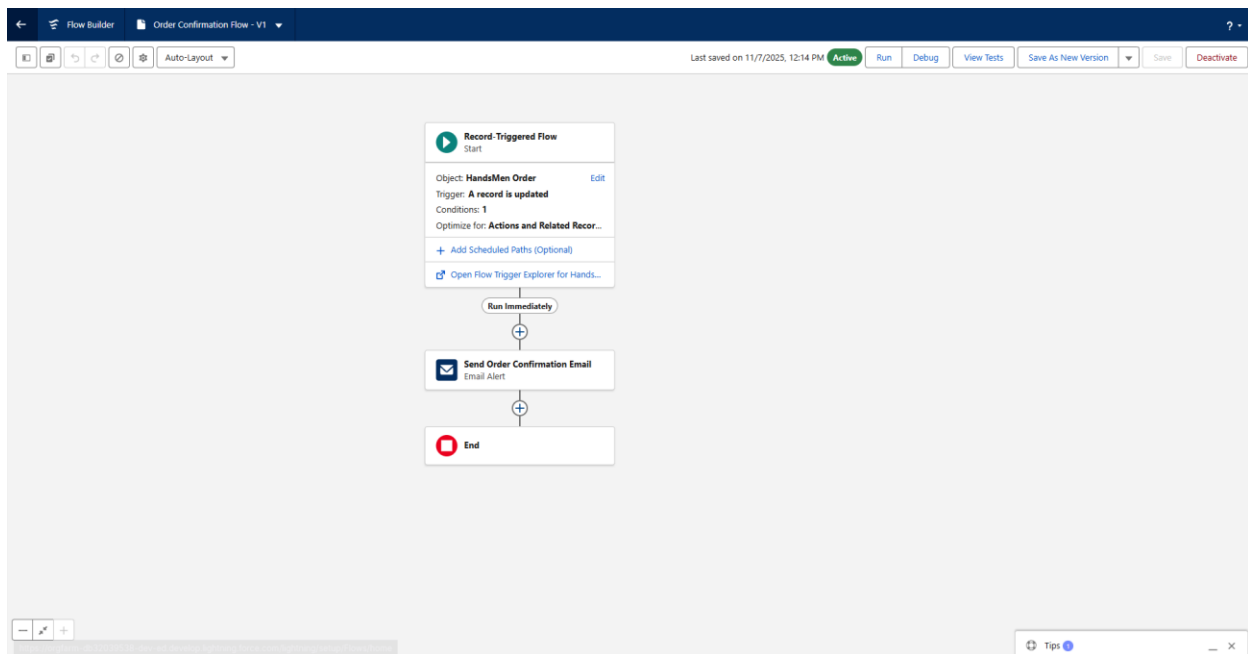
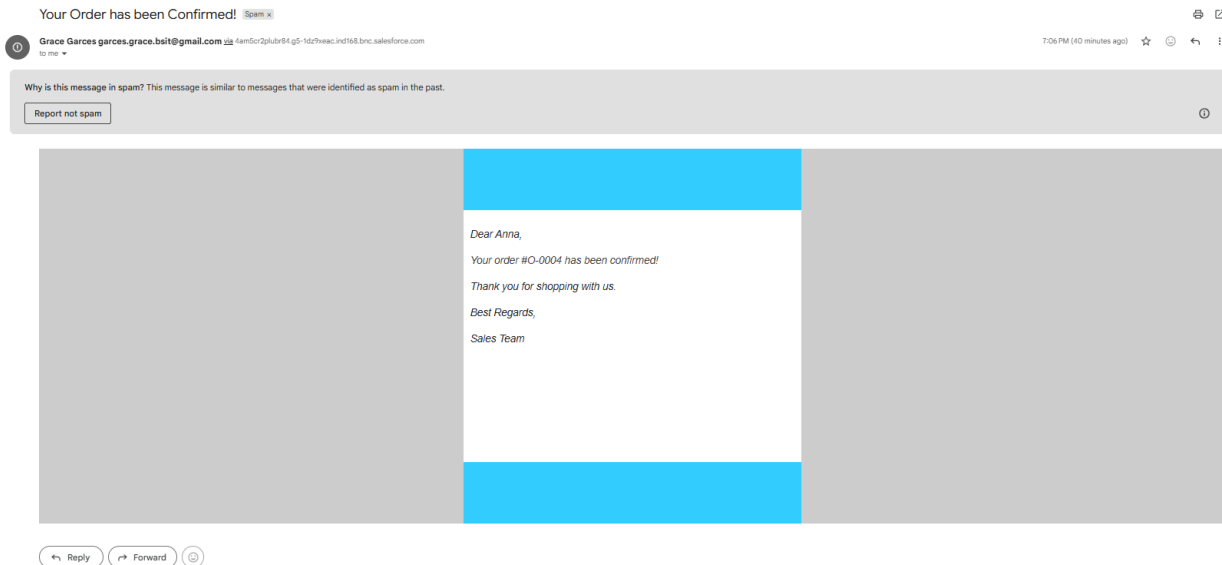


Figure 5.2 Order Confirmation Email



2. Stock Alert Email (Record-Triggered Flow)

Trigger: When Inventory stock quantity drops below the threshold (e.g., 5 units).

Action: Sends an automated alert email to the warehouse or inventory manager.

Purpose: Ensures timely restocking of products and prevents stockouts.

Figure 6.1. Low Stock Alert Email Flow

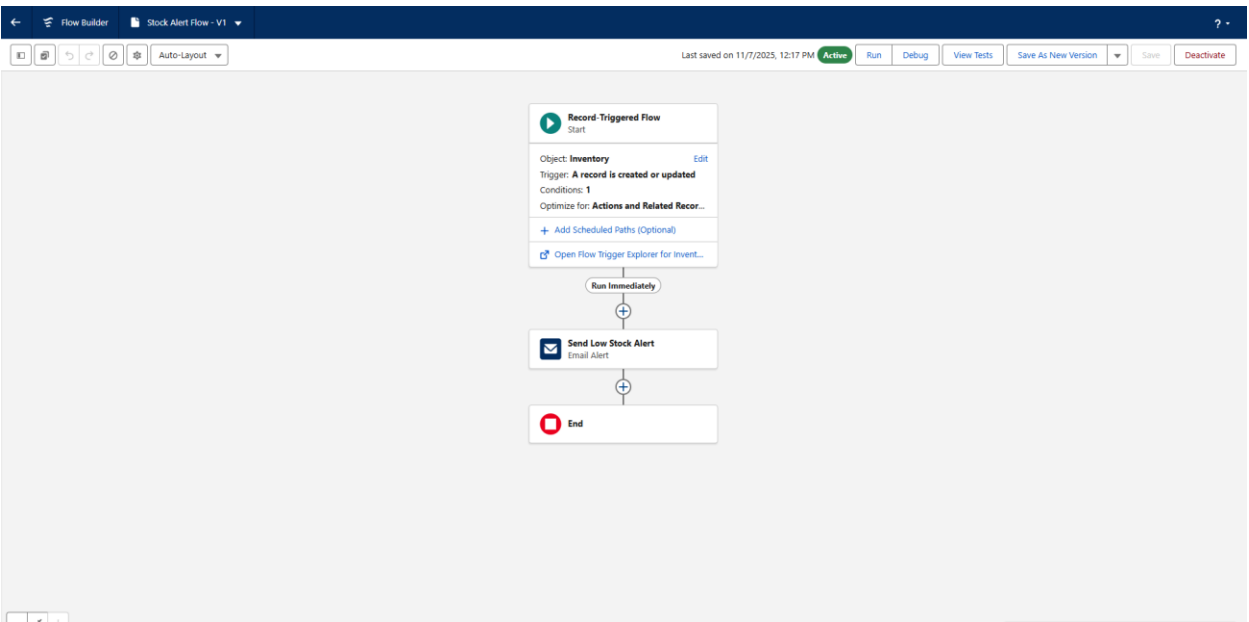
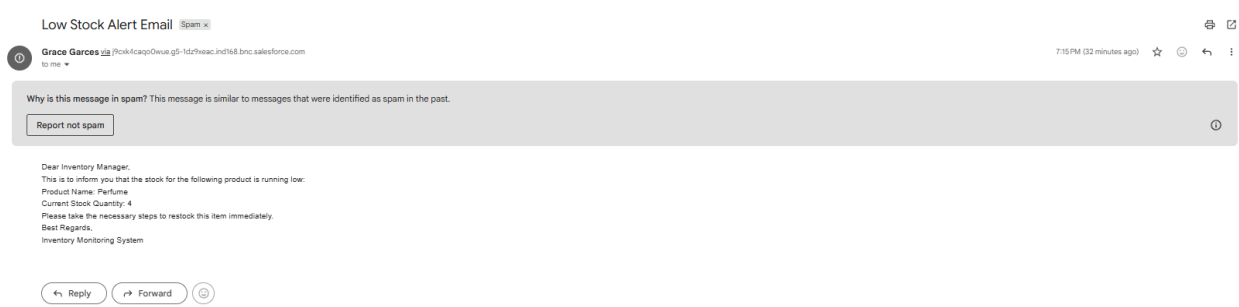


Figure 6.2. Low Stock Alert Email



3. Loyalty Status Update (Scheduled Flow)

Trigger: Scheduled daily flow (runs at midnight).

Action: Updates customer loyalty points and status based on recent purchases and total spending.

Purpose: Maintains up-to-date loyalty tiers, supporting personalized customer rewards.

Figure 7.1. Loyalty Status Update Flow

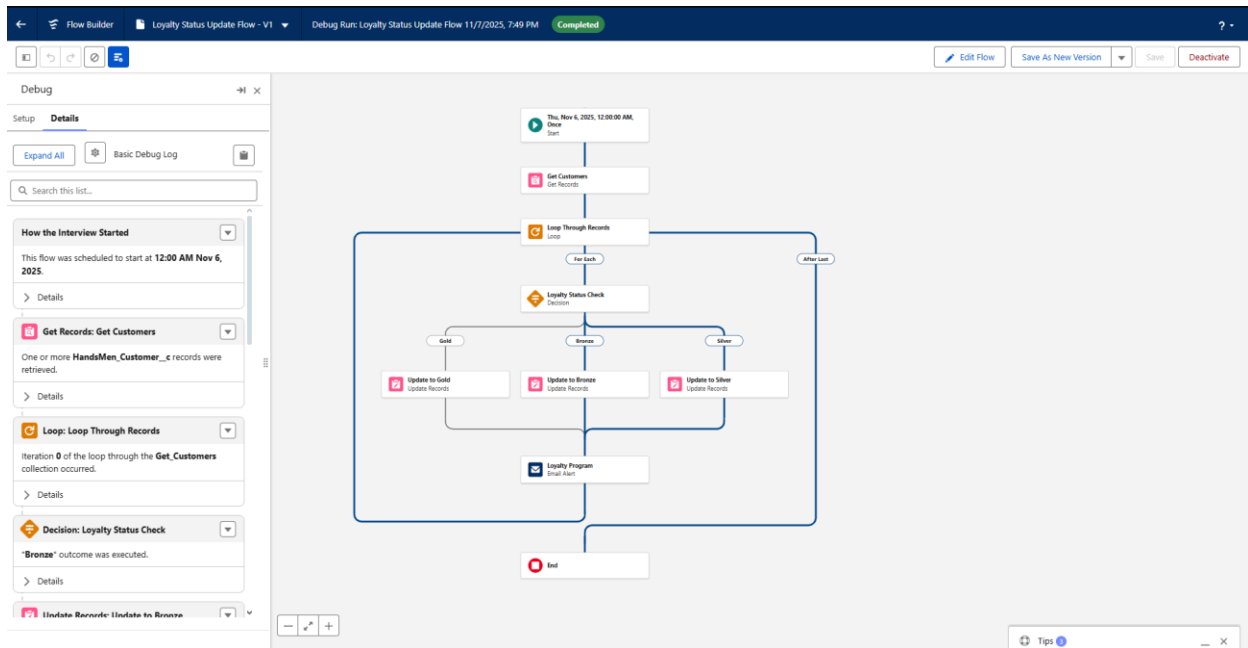
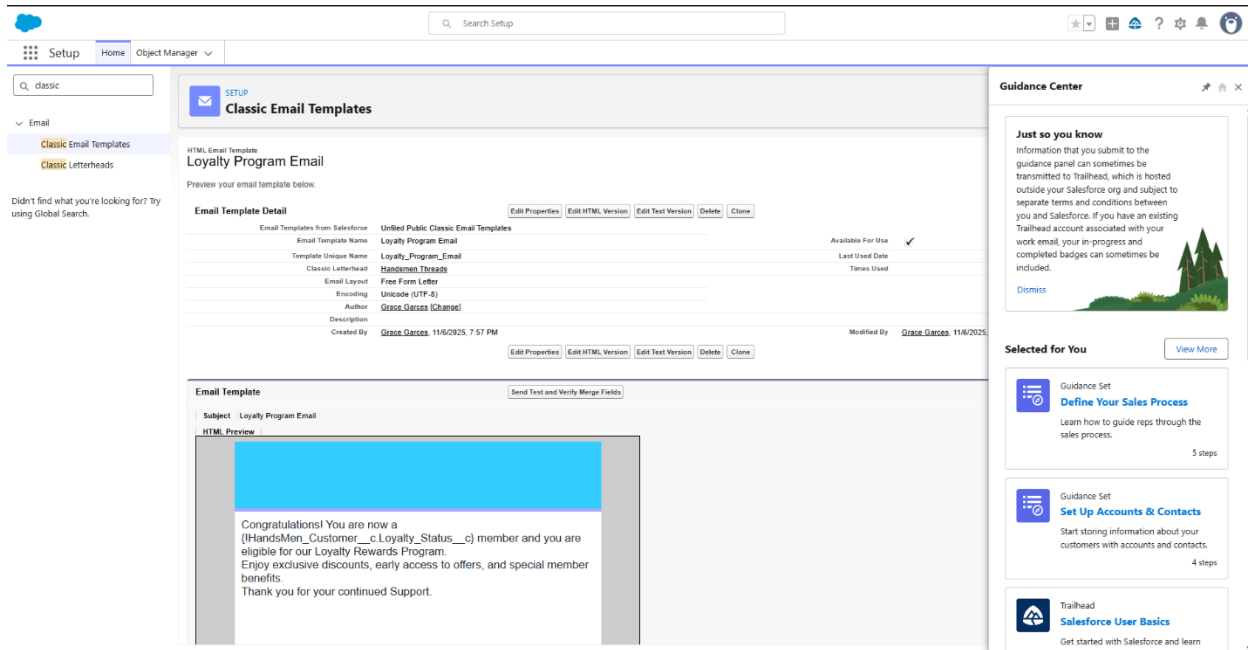


Figure 7.2. Loyalty Classic Email Template



4. Update Order Total

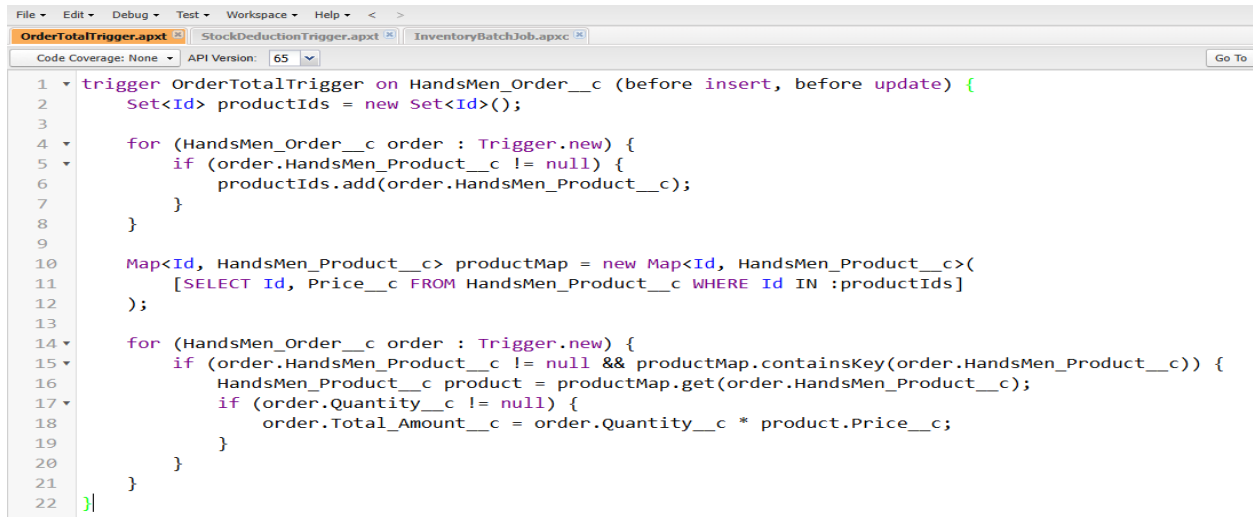
Trigger Name: OrderTotalTrigger

Object: HandsMen_Order__c

Event: Before Insert, Before Update

Purpose: Automatically calculates the total order amount based on the product price and ordered quantity.

Figure 8. Update Order Total Apex Trigger



```
1 trigger OrderTotalTrigger on HandsMen_Order__c (before insert, before update) {
2     Set<Id> productIds = new Set<Id>();
3
4     for (HandsMen_Order__c order : Trigger.new) {
5         if (order.HandsMen_Product__c != null) {
6             productIds.add(order.HandsMen_Product__c);
7         }
8     }
9
10    Map<Id, HandsMen_Product__c> productMap = new Map<Id, HandsMen_Product__c>(
11        [SELECT Id, Price__c FROM HandsMen_Product__c WHERE Id IN :productIds]
12    );
13
14    for (HandsMen_Order__c order : Trigger.new) {
15        if (order.HandsMen_Product__c != null && productMap.containsKey(order.HandsMen_Product__c)) {
16            HandsMen_Product__c product = productMap.get(order.HandsMen_Product__c);
17            if (order.Quantity__c != null) {
18                order.Total_Amount__c = order.Quantity__c * product.Price__c;
19            }
20        }
21    }
22 }
```

This trigger ensures accurate calculation of Total_Amount__c by fetching the product's price and multiplying it by the quantity ordered. It prevents manual errors and ensures pricing consistency.

5. Stock Deduction Trigger

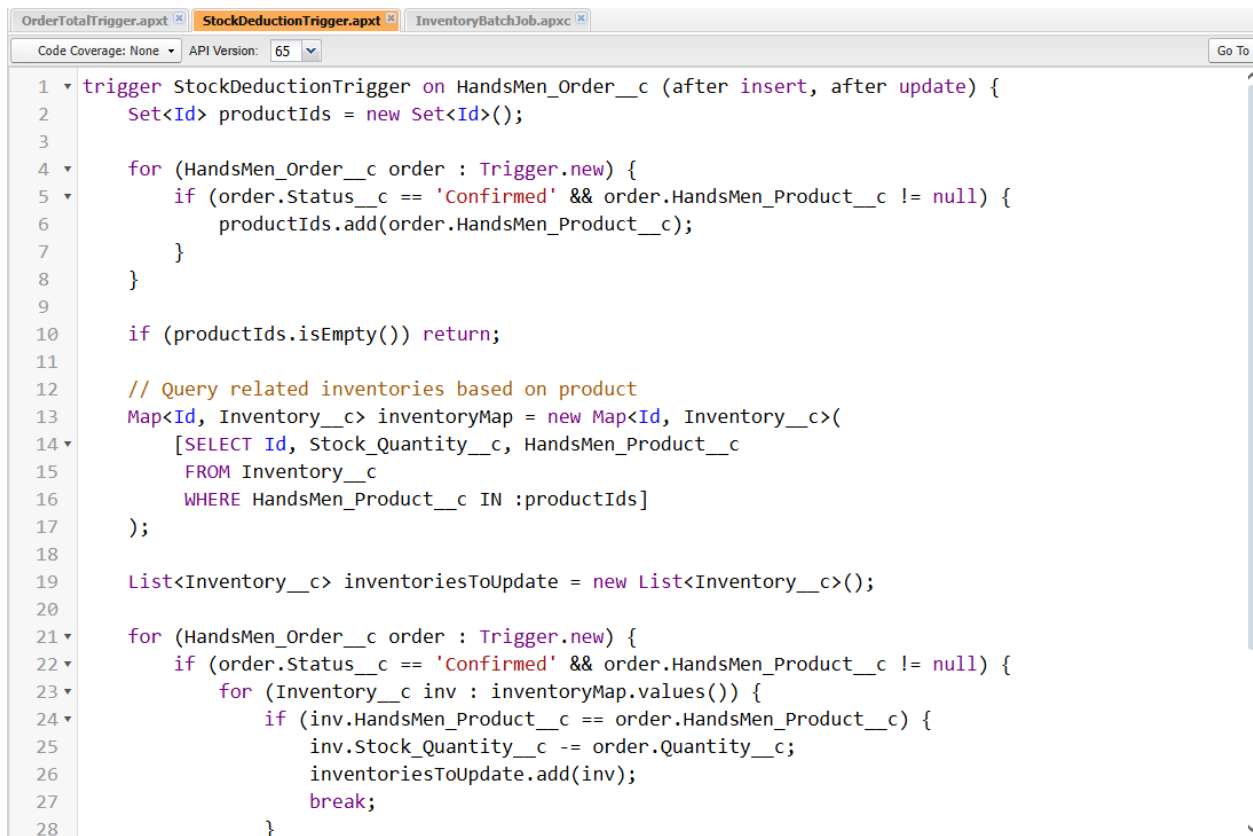
Trigger Name: StockDeductionTrigger

Object: HandsMen_Order__c

Event: After Insert, After Update

Purpose: Automatically deducts stock quantity from inventory when an order is confirmed.

Figure 9. Stock Deduction Apex Trigger



```
1  trigger StockDeductionTrigger on HandsMen_Order__c (after insert, after update) {
2      Set<Id> productIds = new Set<Id>();
3
4      for (HandsMen_Order__c order : Trigger.new) {
5          if (order.Status__c == 'Confirmed' && order.HandsMen_Product__c != null) {
6              productIds.add(order.HandsMen_Product__c);
7          }
8      }
9
10     if (productIds.isEmpty()) return;
11
12     // Query related inventories based on product
13     Map<Id, Inventory__c> inventoryMap = new Map<Id, Inventory__c>({
14         [SELECT Id, Stock_Quantity__c, HandsMen_Product__c
15          FROM Inventory__c
16          WHERE HandsMen_Product__c IN :productIds]
17     });
18
19     List<Inventory__c> inventoriesToUpdate = new List<Inventory__c>();
20
21     for (HandsMen_Order__c order : Trigger.new) {
22         if (order.Status__c == 'Confirmed' && order.HandsMen_Product__c != null) {
23             for (Inventory__c inv : inventoryMap.values()) {
24                 if (inv.HandsMen_Product__c == order.HandsMen_Product__c) {
25                     inv.Stock_Quantity__c -= order.Quantity__c;
26                     inventoriesToUpdate.add(inv);
27                     break;
28                 }
29             }
30         }
31     }
32 }
```

This trigger automatically reduces the stock quantity in the inventory once an order is confirmed. It ensures real-time stock tracking and prevents discrepancies between orders and available inventory.

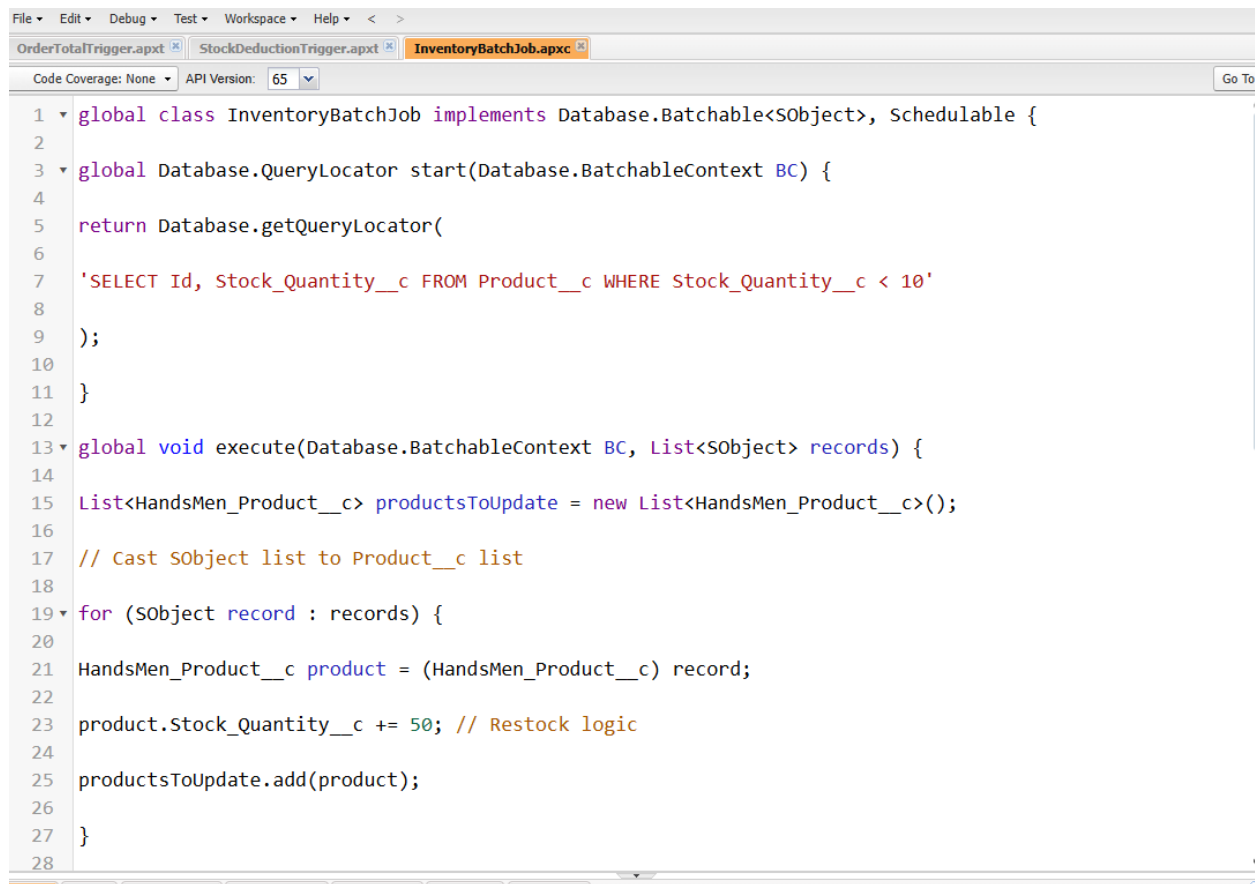
6. Batch Job: InventoryBatchJob

Class Name: InventoryBatchJob

Implements: Database.Batchable<SObject>, Schedulable

Purpose: Automates restocking of products when inventory levels fall below a specified threshold.

Figure 10 Inventory Batch Job Apex Class

The image is a screenshot of a code editor window. At the top, there are tabs for 'OrderTotalTrigger.apxt', 'StockDeductionTrigger.apxt', and 'InventoryBatchJob.apxc', with the last one being the active tab. Below the tabs, there's a menu bar with 'File', 'Edit', 'Debug', 'Test', 'Workspace', and 'Help'. Below the menu bar, there's a status bar showing 'Code Coverage: None' and 'API Version: 65'. The main area of the editor contains the following Apex code:

```
1 global class InventoryBatchJob implements Database.Batchable<SObject>, Schedulable {
2
3     global Database.QueryLocator start(Database.BatchableContext BC) {
4
5         return Database.getQueryLocator(
6
7             'SELECT Id, Stock_Quantity__c FROM Product__c WHERE Stock_Quantity__c < 10'
8
9         );
10    }
11
12    global void execute(Database.BatchableContext BC, List<SObject> records) {
13
14        List<HandsMen_Product__c> productsToUpdate = new List<HandsMen_Product__c>();
15
16        // Cast SObject list to Product__c list
17
18        for (SObject record : records) {
19
20            HandsMen_Product__c product = (HandsMen_Product__c) record;
21
22            product.Stock_Quantity__c += 50; // Restock logic
23
24            productsToUpdate.add(product);
25
26        }
27    }
28}
```

This Apex Batch Class checks all products with a stock quantity lower than 10 and automatically replenishes their stock by 50 units. It improves inventory reliability by preventing shortages and ensuring product availability. The job can also be scheduled to run periodically using Salesforce's Scheduler, ensuring continuous monitoring and restocking without manual intervention.

Phase 5: Testing and Debugging

All components were tested in Salesforce Sandbox using multiple scenarios — valid and invalid data entries, order updates, and automation checks — ensuring reliability and correctness before deployment.

Phase 6: Deployment

Once validated, components (objects, fields, flows, triggers, and templates) were deployed to the production environment using Change Sets.

Project Explanation with Real-World Example

Imagine a customer named John Cruz purchasing a “HandsMen Classic Suit Jacket” from the store. The Sales Associate creates a customer record in Salesforce. A new order is recorded, linked to John and the product he purchased. When the order status is changed to Confirmed, the system automatically:

- Sends John a confirmation email,
- Updates his total purchase amount,
- Deducts the item count in the Inventory object, and
- Re-evaluates his Loyalty Tier based on total spending.

By automating these actions, Salesforce eliminates manual data entry and ensures real-time accuracy.

Testing Approach

A comprehensive testing approach was implemented to ensure that all workflows and automations within the HandsMen Threads Salesforce system functioned as expected. The testing process began with Unit Testing, which validated the logic of Apex Triggers and ensured that field updates, calculations, and record manipulations executed correctly. Next, Integration Testing verified the synchronization between related objects such as Orders, Inventory, and Customers, confirming that data flowed accurately across modules.

Validation Testing was then conducted to ensure that all validation rules properly enforced correct data input and prevented invalid records from being saved. This was followed by Flow Testing, which focused on verifying that all automated processes—such as order confirmation emails, stock alerts, and loyalty status updates—triggered accurately and consistently. Finally, User Acceptance Testing (UAT) was performed by end users to evaluate the system’s usability and ensure that the overall experience was smooth, intuitive, and aligned with business requirements.

Through this multi-layered testing strategy, the team confirmed that all automations, triggers, and workflows performed reliably before deployment to the production environment.

Future Integration

To further enhance HandsMen Threads’ Salesforce ecosystem, several future integrations are planned to expand functionality, improve automation, and elevate customer engagement. These enhancements aim to create a more connected, data-driven, and user-friendly system that supports the company’s long-term growth.

- Integration with Payment Gateways (e.g., Stripe or PayPal) for automatic payment updates.
- Marketing Cloud Integration for targeted campaigns based on loyalty tiers.

- AI-driven Insights using Salesforce Einstein Analytics to predict purchase trends and optimize stock management.
- Mobile App Extension for on-the-go order tracking and customer engagement.

Conclusion

The HandsMen Threads Salesforce project successfully demonstrates how a well-structured CRM solution can transform business operations through automation, data integrity, and customer engagement. By leveraging Salesforce's powerful features—such as Custom Objects, Record-Triggered Flows, and Apex Triggers—the system efficiently manages the company's end-to-end processes, from order creation and inventory tracking to loyalty management and marketing campaigns. The integration of automated workflows significantly reduces manual effort, ensures real-time accuracy, and enhances customer satisfaction through timely communication and personalized experiences.

Furthermore, the project establishes a scalable and adaptable foundation that can evolve with future integrations like payment gateways, AI-driven analytics, and mobile extensions. This solution not only meets the organization's current operational needs but also positions HandsMen Threads for continuous innovation and long-term digital growth within the competitive fashion industry.